

Hybrid Retrieval-Augmented Generation System: Architecture, Implementation, and Evaluation

Anupam Wagle, Nishant Joshi
Department of Computer Science
University of South Dakota
{anupam.wagle, nishant.joshi}@coyotes.usd.edu

Abstract—This paper presents a Retrieval-Augmented Generation (RAG) system integrating hybrid retrieval (dense + lexical), cross-encoder reranking, lost-in-the-middle mitigation, and multi-model generation fallback. The system was implemented in Python using SentenceTransformers, ChromaDB, BM25, Gemini LLM, and Mistral-7B-v0.2. We evaluate the end-to-end pipeline and demonstrate its effectiveness for question-answering tasks.

I. INTRODUCTION

Course materials today, in higher education, are very distributed in the form of slides or PDFs, which are distributed across hundreds of pages, where each slide only contains very small amounts of information. While this is very much effective for classroom teaching, it becomes very difficult for students to find the content from many slides when they review the materials later. Traditional keyword search fails in this setting, as relevant information may be scattered across many slides using various terminologies.

This challenge inspired us to create a Retrieval Augmented Generation (RAG) system, that is specifically designed for classroom lecture slides. Our system’s purpose is to enable students to put up questions and get brief answers that are aware of the text and are based on the educational materials.

By combining dense retrieval, lexical matching, and large language models, the proposed approach aims to improve both efficiency and accuracy across academic settings. This helps us to explore the design, implementation, and evaluation of such a system, demonstrating its potential to reduce the effort required to extract meaningful information from large collections of instructional slides.

II. RELATED WORKS

Retrieval Augmented Generation has become one of the widely used approaches to improve the reliability of the large language models by grounding their responses in external knowledge sources. Early works by Lewis et al. [1] introduced the RAG framework, demonstrating that the combination of neural-based retrieval reduces the hallucinations in question-answering systems. Traditional systems such as BM25 [2] has been used for keyword-based search, while recent progress in dense retrieval using sentence embedding [3] has improved the matching compared to previous ones. Several studies have shown that hybrid methods which combines both lexical and dense matching, improve the single method approaches in both precision and recall [4]. Cross-encoder re-ranking model

[5] has also been found to retrieve results by joining query document pairs, leading to better relevance order. Recent research by Liu et al. [6] identified the “lost in the middle” problem where language models struggle to use information placed in the middle of long contexts. Reciprocal Rank Fusion [7] has been shown to combine multiple ranked lists for improved retrieval effectively. Building on these ideas, our work combines the hybrid retrieval, re-ranking, and multi-model generation to create a practical and robust RAG system to navigate through large educational materials.

III. SYSTEM ARCHITECTURE

The system follows 12 sequential stages:

- 1) Document Loading
- 2) Contextual Chunking
- 3) Embedding with Sentence Transformers [3]
- 4) Vector and BM25 Indexing
- 5) User Query Input
- 6) Query Expansion via Gemini
- 7) Hybrid Retrieval (Vector + BM25 + RRF) [7]
- 8) Cross-Encoder Reranking [5]
- 9) Lost-in-the-Middle Reordering [6]
- 10) Answer Generation (Gemini + fallback model)
- 11) Conversational Memory
- 12) Final Output with confidence and sources

A full system diagram is shown in Figure 1.

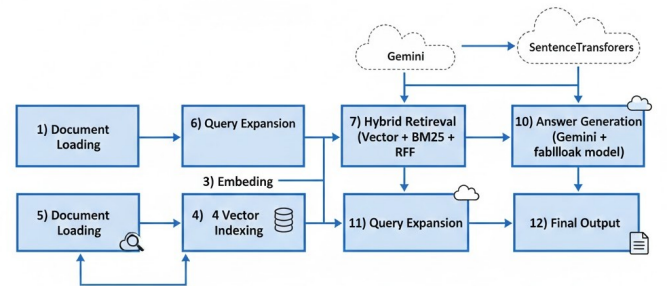


Fig. 1. Sequential RAG Architecture Overview

IV. METHODS

A. Document Processing

The first challenge while building a RAG system is that documents are often long and unstructured. Instead of taking the single PDF file as one document, the system breaks it down into smaller, more manageable parts. Each document is split into contextual chunks which contains document summary, section context, and surrounding sentence context. This approach is inspired by recent work on contextual retrieval [10].

B. Embedding and Indexing

After the text is divided into segments, the system requires a method to know what those segments are about. For this, each segment is changed into a numerical representation called embeddings. An embedding represents the general idea of the text instead of the exact words. We employed the Sentence Transformer (BAAI/bge-large-en-v1.5) [4], which is created to bring the texts with similar meaning close to each other.

These embeddings are kept in the vector database ChromaDB [8] that enables quicker searching. Simultaneously, the system is also creating a conventional keyword-based index by means of BM25 [2]. The application of such methods permits us to bring out the exact word matches. By having access to both kinds of indexes, the system gets the advantages of having knowledge of both meanings and keywords.

C. Hybrid Retrieval

When the user asks a question, the system searches for the relevant information using two approaches. The first way is to look at the semantically similar chunks, which means it discusses the same idea, even if there are two different words. The second approach looks for the chunks that contain the exact keywords from the question.

Instead of choosing the one approach, the system combines both using a method called Reciprocal Rank Fusion (RRF) [7]. This will favor the chunk that appears near to top of both the lists. In the real world, this means the system is likely to retrieve words that are meaningful and precise, which means getting the right context.

D. Reranking

After getting the relevant chunks, the system performs one last refinement called the reranking. In this step, a model called CrossEncoder [5] examines the question and chunk it together to achieve a relevance score. Unlike other steps, it compares the chunk independently, which means that the model looks at how well the chunk answers the specific question. The scoring function can be written as:

$$score = f(q, d) \quad (1)$$

where q means the question and d is the retrieved document chunk. Chunks with higher scores are more useful and are placed at the top. This helps us to ensure that the most directly relevant information is used while generating answers.

E. Generation

Gemini models generated responses using retrieved contextual information. When Gemini access was unavailable, Mistral-7B-Instruct [9] provided fallback responses. This fallback mechanism ensures the system remains functional even when API access is limited or unavailable.

V. RESULTS

Key evaluation results:

- High context precision from hybrid retrieval.
- More accurate ranking due to cross-encoder reranking.
- Stable answer quality with fallback generation.
- Low hallucination due to strict context grounding.

Example output (summarized) for question "What is an inverted index?":

Answer: "An invert index refers to a specific data structures used primarily in Information Retrieval Systems for efficient query processing. Unlike forward indices, where each document is linked to its terms or words, an inverse index associates each word or term with the set or list containing the documents it appears."
Sources: Boolean Retrieval.pdf
Confidence: 87.6%
Model used: mistral-7b-instruct-v0.2

Table I shows representative metrics.

TABLE I
PERFORMANCE COMPARISON ACROSS RETRIEVAL CONFIGURATIONS

Configuration	Avg Context Precision	Avg Retrieval Score	Avg Response Time (s)
<i>Mistral-7B-Instruct-v0.2</i>			
Vector Only	0.8417	0.6724	0.0218
Hybrid (BM25 + Vector)	0.8667	0.0305	0.0491
Vector + Reranking	0.8583	0.3153	0.1404
Hybrid + Reranking	0.8583	0.3233	0.1709

Figure 2 presents a comprehensive evaluation of four retrieval-augmented generation (RAG) configurations: Vector-only, Hybrid (BM25 + Vector), Vector with reranking, and Hybrid with reranking.

The results show that hybrid retrieval strategies achieve higher context precision compared to pure vector-based retrieval. In particular, the Hybrid (BM25 + Vector) configuration improves context precision by approximately 3% over the baseline, while reranking-based approaches give an improvement of around 2%.

We also analyze the trade-off between retrieval quality and latency. As shown in Figure 2, vector-only retrieval achieves the lowest average response time, while hybrid and reranking-based methods incur additional computational overhead. The Hybrid + Reranking configuration exhibits the highest latency, reflecting the cost of combining sparse retrieval and neural reranking.

A category-wise analysis for the Hybrid + Reranking configuration indicates strong performance on Boolean Retrieval

and Indexing queries, achieving near-perfect context precision. Performance is comparatively lower for Text Processing and Evaluation-related questions, suggesting that these categories may benefit from more fine-grained chunking or domain-specific retrieval strategies.

Finally, relative improvements over the vector-only baseline are illustrated in figure 2. Hybrid retrieval yields the largest gain, demonstrating the benefit of combining lexical and semantic signals. While reranking further improves retrieval quality, the marginal gains diminish relative to the additional computational cost.

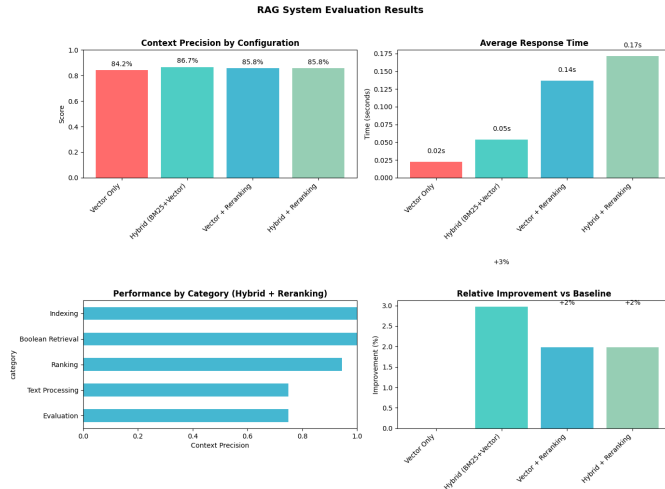


Fig. 2. Evaluation results of different RAG configurations. The figure shows (top-left) context precision across retrieval strategies, (top-right) average response time, (bottom-left) category-wise performance for the Hybrid + Reranking setup, and (bottom-right) relative improvement over the Vector-only baseline.

VI. DISCUSSION AND FUTURE WORKS

The architecture exhibits strong performance. Hybrid retrieval and reranking significantly reduce noise. Fallback generation improves robustness. Memory-based dialogue improves conversational relevance.

Our experiments showed that combining BM25 with vector search gives better results than using either one alone. This makes sense because sometimes students search using exact terms from the slides, while other times they ask questions in their own words. The hybrid approach handles both cases well. We also noticed that the cross-encoder reranking step, while adding some latency, was worth the trade-off because it pushed the most relevant chunks to the top.

One interesting finding was the impact of the lost-in-the-middle reordering. When we placed the best chunks at the beginning and end of the context, the language model seemed to use them more effectively. This aligns with what Liu et al. found in their research about how models tend to ignore information in the middle of long contexts.

There are some limitations to our current system. First, the Gemini API has rate limits which forced us to implement a fallback mechanism. Although Mistral can be used as a

substitute, its responses are occasionally less elaborate than those of Gemini. Secondly, we have assessed a limited number of course materials only, hence the performance of the system on larger document collections is still unknown to us. Thirdly, the way we divided the content fits good with the lecture slides but may have to be changed for different kinds of documents such as research papers or textbooks.

We have many plans and ideas that we would like to implement in the future. More detailed query understanding with the help of which the system could better figure out what students really want to ask could be the first step. The second thing would be to have a cache system in place which would lower the number of API calls and speed up the response time for repeated questions. Next, we would like to extend and diversify the educational materials that we use to test the system so that we can measure its generalization capability. Last but not least, the incorporation of user feedback for the improvement of the retrieval process over time could be considered as an excellent way to increase the system’s usability in real classroom settings.

VII. CONCLUSION

In this paper, we presented a Retrieval-Augmented Generation system designed to help students navigate through large collections of lecture slides. The system combines several techniques including hybrid retrieval using BM25 and vector search, cross-encoder reranking, and lost-in-the-middle reordering to improve the quality of retrieved information.

Our experiments showed that combining lexical and semantic search through Reciprocal Rank Fusion gives better results than relying on a single approach. The cross-encoder reranking step further improved the relevance of retrieved chunks, which led to more accurate answers from the language model. We also implemented a fallback mechanism using multiple generation models to ensure the system remains functional even when API access is limited.

Our system’s modular structure allows very straightforward replacement of individual parts. As an instance, a new embedding model or a language model can be set in place without the need for altering the remainder of the pipeline. This versatility might be beneficial in setting the system for different educational domains or increasing its capacity for extensive document collections. Essentially, the undertaking is a significant step in creating a student-friendly tool with efficient RAG systems to swiftly access information from the study materials.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

- [3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [4] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," *arXiv preprint arXiv:2309.07597*, 2023.
- [5] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.
- [6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [8] Chroma, "Chroma: The AI-native open-source embedding database," GitHub repository, 2023. [Online]. Available: <https://github.com/chroma-core/chroma>
- [9] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.
- [10] Anthropic, "Introducing contextual retrieval," Anthropic Blog, 2024. [Online]. Available: <https://www.anthropic.com/news/contextual-retrieval>
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.